

Glide 图片加载库：核心特性、基础使用、关键功能及核心原理讲解

Glide 是 Android 开发中最常用的图片加载库之一（由 Google 推荐），主打**高效、易用、轻量**，支持图片/动图加载、缓存管理、生命周期绑定等核心能力，能极大简化图片加载的开发流程。下面从「核心特性、基础使用、关键功能、核心原理（极简）」四个维度讲解，新手也能快速上手。

一、核心特性（先知道它能干嘛）

1. **全类型加载**：支持网络图片、本地图片、资源文件（drawable/mipmap）、Uri、字节流等；
2. **生命周期绑定**：自动和 Activity/Fragment 生命周期联动，避免内存泄漏；
3. **智能缓存**：默认实现「内存缓存 + 磁盘缓存」，可自定义缓存策略；
4. **动图支持**：原生支持 Gif 加载，无需额外配置；
5. **图片处理**：内置裁剪、缩放、圆角、模糊等变换能力；
6. **性能优化**：自动处理图片解码、内存复用，降低 OOM 风险。

二、基础使用（三步上手）

1. 集成 Glide（最新版以官方为准）

在 `app/build.gradle`（Module 级别）中添加依赖：

Code block

```
1 dependencies {
2     // 核心库（必加）
3     implementation 'com.github.bumptech.glide:glide:4.16.0'
4     // 注解处理器（用于自定义 GlideModule，可选但建议加）
5     annotationProcessor 'com.github.bumptech.glide:compiler:4.16.0'
6 }
```

权限配置（加载网络图片需加）：

在 `AndroidManifest.xml` 中添加网络权限：

Code block

```
1 <uses-permission android:name="android.permission.INTERNET" />
2 <!-- Android 9.0+ 需允许明文网络请求（若图片地址是 http 而非 https） -->
```

```
3 <application
4     ...
5     android:usesCleartextTraffic="true">
6 </application>
```

2. 最简单的加载示例

Glide 核心 API 是 `Glide.with()` 链式调用，一行代码即可加载图片到 `ImageView`：

Code block

```
1 // Java 示例
2 Glide.with(this) // 上下文 (Activity/Fragment/View 均可, 自动绑定生命周期)
3     .load("https://xxx.com/xxx.jpg") // 图片地址 (网络/本地/资源)
4     .into(imageView); // 目标 ImageView
5
6 // Kotlin 示例 (语法更简洁)
7 Glide.with(this)
8     .load("https://xxx.com/xxx.jpg")
9     .into(imageView)
```

3. 加载不同来源的图片

Code block

```
1 // 1. 加载本地文件 (File 对象)
2 Glide.with(this).load(new File("/sdcard/xxx.jpg")).into(imageView);
3
4 // 2. 加载资源文件 (drawable)
5 Glide.with(this).load(R.drawable.ic_xxx).into(imageView);
6
7 // 3. 加载 Uri (比如相册选择的图片)
8 Glide.with(this).load(uri).into(imageView);
```

三、常用关键功能（解决实际开发需求）

1. 占位图 & 错误图

加载过程中显示占位图，加载失败显示错误图：

Code block

```
1 Glide.with(this)
2     .load("https://xxx.com/xxx.jpg")
3     .placeholder(R.drawable.placeholder) // 加载中占位图
4     .error(R.drawable.error) // 加载失败图
```

```
5     .fallback(R.drawable.fallback) // 加载地址为 null 时显示的图
6     .into(imageView);
```

2. 图片变换（裁剪、圆角、缩放）

Glide 内置 `Transformation` 处理图片，常用场景：

Code block

```
1  import com.bumptech.glide.load.resource.bitmap.CenterCrop;
2  import com.bumptech.glide.load.resource.bitmap.RoundedCorners;
3
4  // 1. 圆角图片（参数：圆角半径，单位 dp 需转 px，或直接传 px）
5  int radius = dp2px(8); // 自定义 dp 转 px 方法
6  Glide.with(this)
7      .load("https://xxx.com/xxx.jpg")
8      .transform(new RoundedCorners(radius)) // 圆角
9      .into(imageView);
10
11 // 2. 圆形图片（需依赖 glide-transformations 库）
12 // 先添加依赖：implementation 'jp.wasabeef:glide-transformations:4.3.0'
13 import jp.wasabeef.glide.transformations.CropCircleTransformation;
14 Glide.with(this)
15     .load("https://xxx.com/xxx.jpg")
16     .transform(new CropCircleTransformation()) // 圆形
17     .into(imageView);
18
19 // 3. 缩放裁剪（CenterCrop / FitCenter）
20 Glide.with(this)
21     .load("https://xxx.com/xxx.jpg")
22     .centerCrop() // 居中裁剪（铺满 ImageView，超出部分裁剪）
23     // .fitCenter() // 自适应（完整显示图片，不裁剪，可能留空白）
24     .into(imageView);
```

3. 缓存控制（自定义缓存策略）

Glide 默认缓存规则：内存缓存优先 → 磁盘缓存 → 网络请求，可手动调整：

Code block

```
1  Glide.with(this)
2      .load("https://xxx.com/xxx.jpg")
3      .skipMemoryCache(true) // 跳过内存缓存（默认 false）
4      .diskCacheStrategy(DiskCacheStrategy.NONE) // 跳过磁盘缓存
5      // 磁盘缓存策略可选值：
6      // - NONE：不缓存
7      // - ALL：缓存原图 + 处理后的图
```

```
8      // - SOURCE: 只缓存原图
9      // - RESULT: 只缓存处理后的图 (默认)
10     .into(imageView);
11
12     // 清理缓存 (需在子线程执行)
13     new Thread(() -> {
14         Glide.get(context).clearDiskCache(); // 清理磁盘缓存
15     }).start();
16     Glide.get(context).clearMemory(); // 清理内存缓存 (主线程执行)
```

4. 加载动图 (Gif)

Code block

```
1  // 加载 Gif (自动识别, 无需额外配置)
2  Glide.with(this)
3      .load("https://xxx.com/xxx.gif")
4      .into(imageView);
5
6  // 强制加载为静态图 (Gif 只显示第一帧)
7  Glide.with(this)
8      .asBitmap() // 强制转 Bitmap
9      .load("https://xxx.com/xxx.gif")
10     .into(imageView);
11
12  // 控制 Gif 播放次数
13  Glide.with(this)
14      .asGif()
15      .load("https://xxx.com/xxx.gif")
16      .listener(new RequestListener<GifDrawable>() {
17          @Override
18          public boolean onLoadFailed(@Nullable GlideException e, Object model,
19              Target<GifDrawable> target, boolean isFirstResource) {
20              return false;
21          }
22          @Override
23          public boolean onResourceReady(GifDrawable resource, Object model,
24              Target<GifDrawable> target, DataSource dataSource, boolean isFirstResource) {
25              resource.setLoopCount(1); // 播放 1 次 (默认无限循环)
26              return false;
27          }
28      }).into(imageView);
```

5. 生命周期控制

`Glide.with()` 传入的上下文决定了生命周期：

- 传 `Activity/Fragment`：图片加载会随组件销毁自动停止，避免内存泄漏；
- 传 `ApplicationContext`：不绑定生命周期，需手动控制（慎用）。

四、核心原理（极简版，知其然知其所以然）

1. **请求流程**：`with()` 绑定生命周期 → `load()` 构建请求 → `into()` 执行请求（先查内存缓存 → 磁盘缓存 → 网络请求）；
2. **缓存分层**：
 - 内存缓存：`LruCache` + 弱引用，优先快速获取；
 - 磁盘缓存：分为「原图缓存」和「处理后缓存」，减少重复解码；
3. **线程调度**：网络请求/磁盘读取在子线程，图片解码/UI 更新在主线程，通过 `Handler` 切换；
4. **图片复用**：复用 `Bitmap` 池，减少内存分配和 GC，降低 OOM 风险。

五、避坑点

1. **Context 泄漏**：避免在非生命周期组件（如单例）中使用 `ApplicationContext` 加载图片，或忘记停止请求；
2. **OOM 风险**：加载超大图片时，建议指定图片尺寸（`.override(width, height)`），避免解码原图；
3. **混淆配置**：若开启混淆，需在 `proguard-rules.pro` 中添加 `Glide` 混淆规则（官方文档有现成模板）。

总结

`Glide` 的核心是「极简的链式 API + 智能的生命周期/缓存管理」，日常开发中，基础加载 + 占位图 + 简单变换就能满足 80% 的需求，复杂场景（如自定义解码器、图片压缩）可参考官方文档扩展。

官方文档：<https://bumptech.github.io/glide/>

（注：文档部分内容可能由 AI 生成）